

Capítulo 4: Memória Cache.

Métodos de Acesso

- **Acesso Sequencial:** O acesso é feito em uma sequência linear específica. Usa-se um mecanismo compartilhado leitura-escrita que precisa ser movido do local atual para o desejado passando e repetindo cada registro intermediário: **VHS, fita de dados...**
- **Acesso Direto:** Envolve o mecanismo de leitura-escrita compartilhado. Porém, os blocos ou registros individuais têm um endereço exclusivo, baseado no local físico. O acesso é feito diretamente porém ao entrar na trilha é feito da maneira sequencial: **HDs.**
- **Acesso Aleatório:** Cada local endereçável na memória possui endereço exclusivo. Qualquer local pode ser selecionado aleatoriamente e endereçado diretamente. O tempo de acesso é igual para todos.
- **Acesso Associativo:** É o tipo de memória de acesso aleatório que permite fazer uma comparação de um certo número de bit desejados dentro de uma palavra para uma combinação específica. **Memória Cache.**

Desempenho.

Latência é o tempo para se realizar a leitura para escrita

Tempo de ciclo de memória: é o tempo de acesso + qualquer tempo adicional antes de um segundo

Taxa de Transmissão: taxa em que os dados podem ser transferidos para dentro ou fora.

Hierarquia de Memória.

- Tempo de acesso mais rápido, mais custo por bit.
- Maior capacidade, menor custo por bit.
- Maior capacidade, tempo de acesso mais lento.

Cache

Geralmente não é visível ao processador, ela basicamente organiza a movimentação de dados entre a memória principal e os registradores do processador. Ela é uma expansão **tilibra** da memória principal

Princípios da memória cache.

- A cache contém uma cópia de partes da memória principal.
- Se o processador tenta ler uma palavra e ela está na cache, ela é entregue para o processador.
- Se não, a memória principal manda para cache, e depois para o processador.

Pelo princípio da localidade, o que foi acessado em curto tempo, pode ser acessado de novo. Por isso manda para memória cache.

A cache consiste em M blocos, chamados linhas e cada linha contém K palavras e mais uma tag de alguns bits.

A tendência é que quanto maior o tamanho da cache ela tende a ser mais lenta.

Função de Mapeamento

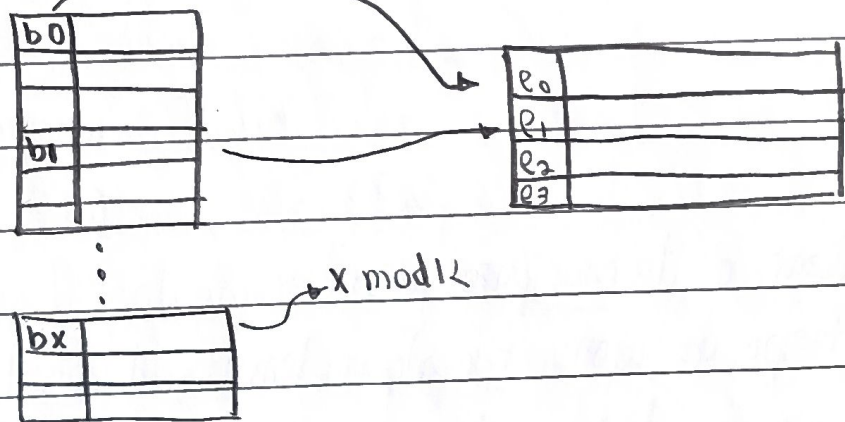
Existem pois tem menos linhas de cache do que blocos da memória principal.

Mapeamento Direto.

Mapeia cada bloco da memória principal a apenas uma linha de cache possível.

A técnica é simples e pouco dispendiosa para se implementar.

Porém a desvantagem de todos os blocos possuem um local de cache fixo, que se um programa referenciar dois blocos diferentes, que estão na mesma linha, eles serão continuamente trocados.

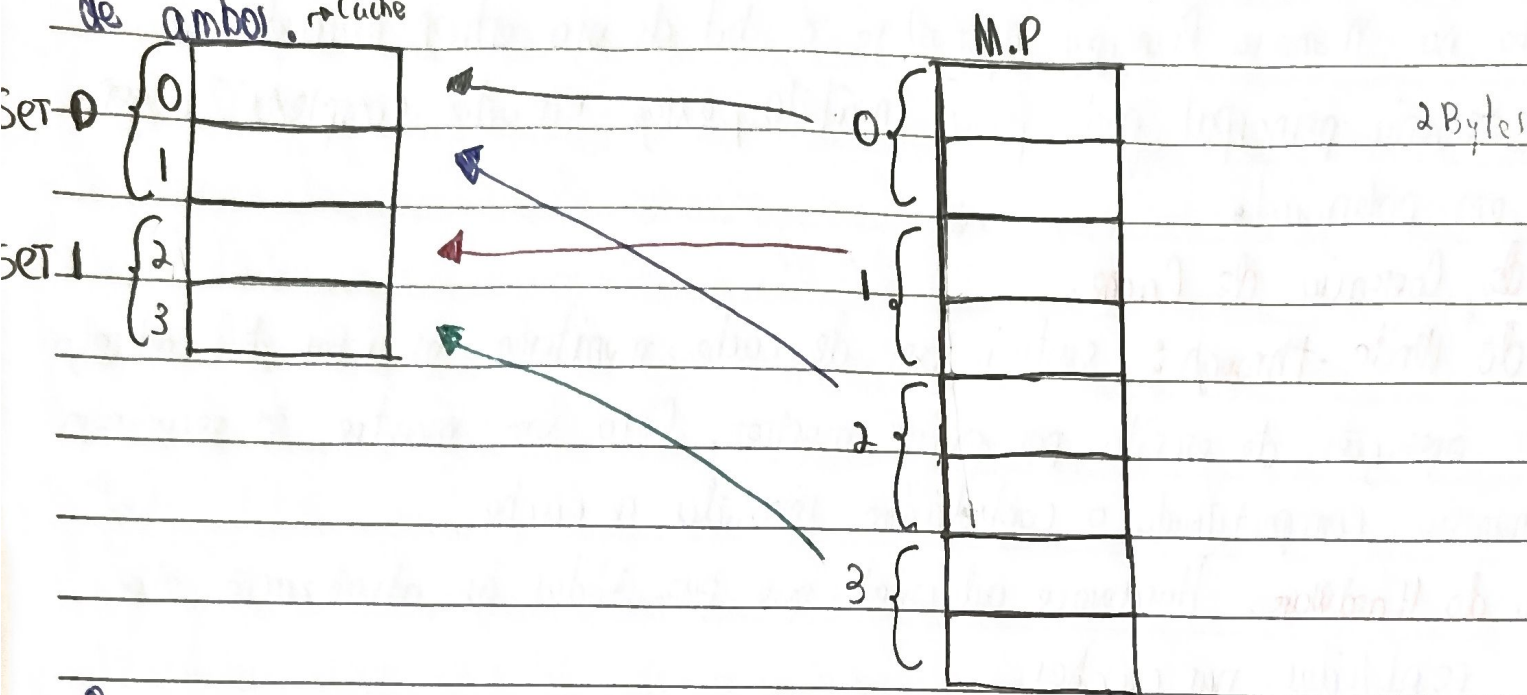


Mapeamento Associativo:

Permite que cada bloco da memória principal seja carregado em qualquer linha de cache. Basicamente os blocos são alocados em qualquer lugar da cache, porém quando a cache está cheia é utilizado um algoritmo de substituição que decide qual linha será ocupada.

Mapeamento Associativo por Conjunto.

O (set associative) pega as melhores partes do associativo/direto, e reduz as desvantagens de ambos. → Cache



O algoritmo de substituição é utilizado para saber qual dos dois de um set será removido.

Algoritmo de Substituição

São utilizados nos mapeamento: Associativo // Associativo por Conjunto.

Tem como princípio em uso quando a cache está cheia e ~~precisa~~ um novo bloco precisa ser utilizado. Portanto existem alguns

Usado menos recentemente (LRU): Talvez o mais eficaz

Primeiro a entrar, primeiro a sair: (FIFO)

Usado menos frequentemente (LFU)

Aleatório: Oferece um desempenho ligeiramente inferior, porém muito simples.

Política de Escrita

Se um bloco antigo na cache não foi alterado, ele não precisa atualizar para substituir.

Seja, caso ocorra qualquer alteração é necessário atualizar antes de substituir.

Problemas • Palavra foi atualizada apenas cache, então a palavra da M.P. é inválida.

• Palavra foi atualizada apenas na MP, a cache então é inválida.

Solução

Write-Through: As atualizações são feitas tanto na memória cache/principal.

A desvantagem dessa técnica é existe um grande tráfego na memória o que pode causar um gargalo.

Write-Back: As atualizações são feitas apenas na cache. Quando ocorre alguma modificação, um bit de uso é marcado (associado a linha), quando o bloco é substituído ele é escrito na Memória Principal apenas se o bit de uso estiver marcado.

- Parte da memória principal pode ficar inválida, exige circuitos complexos e gera um gargalo em potencial.

Técnicas de Coerência de Cache.

Observação do Write-Through: controladores de cache monitoram as linhas de endereço para detectar operações de escrita por outros mestres. Caso um mestre escrever num local de memória compartilhada, o controlador invalida a cache.

Transparência do Hardware: Hardware adicional que faz todas as atualizações da M.P serem replicadas nas caches.

Memória não cacheável: Apenas uma parte da memória principal é compartilhada por mais de um processador, e não é mantida na cache.

Tamanho da linha

- Blocos maiores reduzem o número de blocos que cabem na cache.

↳ Resultando em blocos sendo constantemente trocados, por não ter muitos na cache.

- Quanto maior o bloco, mais distante as palavras ficam em relação a elas

↳ Fode o princípio da localidade

Número de memórias Cache.

Assim que a densidade lógica aumenta, torna-se possível uma cache no mesmo chip que o processador.

↳ Que reduz a atividade do barramento, agiliza o tempo de execução de desempenho e deixa o barramento livre para outra ação.

A organização de cache mais comum é com a cache L1, interna e a cache L2 externa, existe há necessidade da cache L2 pois assim quando a informação não estiver na L1, a L2 poderá buscar mais rapidamente.

É comum dividir a cache do mesmo nível em duas funções: uma dedicada a instruções e a outra a dados.

Capítulo 4: Memória Cache

Perguntas Revisão

4.1 = Acesso sequencial: É um tipo de acesso em que tudo ocorre de maneira linear.

Ou seja, quando se deseja acessar tudo ocorre da maneira sequencial.

Acesso Direto: O acesso é feito diretamente no local em que se deseja e logo depois ele ocorre de maneira sequencial. Portanto, cada bloco ou registro possui os endereços exclusivos que são escolhidos diretamente e depois ~~passa~~ a trilha é acessada de maneira individual.

Acesso Aleatório: Cada local possui um endereço exclusivo e o acesso aleatório escolhe um aleatoriamente e a trilha escolhida é acessada diretamente.

4.2

Tempo de Acesso, custo e capacidade

A relação entre essas três palavras é bem simples.

Maior capacidade, maior tempo de acesso // Menor capacidade, menor tempo de acesso.
menor custo L maior custo

4.3

O princípio da localidade se relaciona com o uso de múltiplos níveis de memória pois:

- Porque se tem o bloco da M.P. ao invés do que se quer propriamente.

↳ Princípio da localidade, dados próximos tem a tendência de ser acessados.

Ou seja é mais do que necessário os múltiplos níveis que consegue acessar de maneira mais rápida e que deseja pela distância da cache até o processador e por ser de maior velocidade. Portanto ao se colocar o que mais acessa em cada cache você usa o princípio de coisas próximas tem tendência de serem acessadas de novo.

4.4

Mapeamento Direto: Ele basicamente é composto por tag, linha, conjunto.

Nele, todos dados já tem seu próprio endereço. Pois é realizado um cálculo de divisão onde o resto é a posição. É uma técnica simples e que oferece como vantagem o fato de cada endereço ser de local fixo.

Mapeamento Associativo: Neste mapeamento, é composto apenas por tag, palavra.

Nele, os dados são alocados onde tem espaço ou seja qualquer linha.

Portanto, quando todas estão ocupadas ele usa de um algoritmo de substituição para escolher qual se vai.

Mapeamento Associativo em Conjunto: Esse mapeamento é composto por tag, conjunto, palavra.

O funcionamento dele é basicamente a união dos dois anteriores.

Nele os locais são previamente definidos pelo o cálculo de mod, que define em qual "set" eles serão alocados. Porém quando está cheio, é utilizado dentro do set a qual o bloco pertence em que a linha do set que ele ficará é escolhida por meio de um algoritmo de substituição.

4.5

Tag, linha, palavra.

Tag: Identifica um dos blocos que podem caber em linha.

Linha: Identifica a linha da cache.

Palavra: Basicamente são os dados, o byte ou palavra do bloco da M.P.

4.6

Tag, palavra

Tag: identifica um dos blocos que podem caber em linha

Palavra: É basicamente o byte ou palavra da identificável dentro do bloco da M.P.

4.7

Tag, Conjunto, Palavra

Tag: Identifica um dos blocos que podem caber em set

Conjunto: Identifica um dos conjuntos da cache

Palavra: Byte ou palavra identificável dentro do bloco da M.P.

4.8

Localidade Espacial: É um princípio que diz que valores armazenados próximos tem a tendência de serem acessados.

Localidade Temporal: É um princípio que diz que as coisas recentemente acessadas tem tendência de serem acessadas novamente.

4.9

O uso de múltiplos níveis de memória é uma das melhores maneiras de se explorar ambos princípios, exatamente por colocar blocos dentro dos níveis de cache.

Problemas

4.1 - Memória Principal é composta por: $2^2 \times 2^{10} \times 2^7$ palavras $\rightarrow 2^{19}$ 19 bits

Cache: Conjuntos de 4 linhas e um total 64 linhas $\frac{2^6}{2^2} \rightarrow 2^4 \rightarrow 16$ conjuntos

TAG = Número bits total - Conjunto - Palavra.

$$19 - 4 - 7 = 8$$

Conjunto: Para escrever qual conjunto é: 16 $\rightarrow 2^4$ 4 bits.

Palavra: Haja vista que cada bloco tem 128 $\rightarrow 2^7$ 7 bits

TAG = 8	CONJUNTO = 4	PALAVRA = 7
---------	--------------	-------------

4.2

Cache: 2 linhas por conjunto: Tam total = 8K $\rightarrow 2^3 \cdot 2^{10} \rightarrow \frac{2^{13}}{2^4} \rightarrow 2^9 \rightarrow 2^8$

Memória Principal 64M $\rightarrow 2^6 \cdot 2^{20} \rightarrow 2^{26} \rightarrow 26$ bits

$$\text{TAG} = 26 - 8 - 4 = 14$$

Conjunto = 8

Palavra = 4

4.3

a)

4.5 - A memória principal possui 32 bits

A cache tem um total de 16K $\rightarrow 2^4 \times 2^{10} = 2^{14}$

Tamanho da linha 32 bits $\times 4 = 128$ bits que é 16 bytes = $2^4 // 4$ bits para palavra estão

Portanto o número de linhas seria $2^{14} / 2^4 \rightarrow 2^{10}$ linhas //

Como cada conjunto tem 4 linhas $2^{10} / 2^2 \rightarrow 2^8 // 8$ bits para conjunto

$$\text{TAG} = 32 - 8 - 4 = 20$$

TAG = 20	CONJUNTO = 8	PALAVRA = 4
----------	--------------	-------------

tilibra

bits // Tem linha 2×16 bits = 32 bits = 4 bytes - 1

total $\rightarrow 2^{14} \rightarrow 2^{12}$ linhas $\rightarrow 2^{10}$ conjuntos
 2^2 linhas/conjunto

16 bits = 2 bytes = 1 bit para representar

-13

TO = 10	PALAVRA = 1
---------	-------------

$2^2 \rightarrow 2^{15}$

$$TAG = 18 - 9 - 2 = 7$$

equivale a 2 bytes = 1 bit.

bloco cabe 4 palavras

512 conjuntos $\rightarrow$ 9 bits para conjunto 2

conjunto

palavra

2^{32} ou seja 32 bits para endereçar.

Palavras endereçamento 11 bits.

no conjunto = 16 conjuntos.

4.6 4K de palavras de 32 bits // Tem linha 2x16 bits - 32 bit = 4 bytes - 1
 $2^2 \times 2^{10} \times 2^2 \rightarrow 2^{14}$ total $\rightarrow \frac{2^{14}}{2^2} \rightarrow 2^{12}$ linhas $\rightarrow 2^{10}$ conjuntos
 4 1K Bytes $\frac{2^{14}}{2^2}$ linha/conjunto

Portanto cada palavra é de 16 bits - 2 bytes = 1 bit para representar

$$TAG = 2^4 - 10 - 1 = 13$$

TAG = 13	CONJUNTO = 10	PALAVRA = 1
----------	---------------	-------------

4.10

Memória Principal: $2^6 \times 2^{10} \times 2^2 \rightarrow 2^{18}$

$$TAG = 18 - 9 - 2 = 7$$

4096 palavras: cada palavra equivale a 2 bytes = 1 bit.

Conjunto de 2 blocos e cada bloco cabe 4 palavras

4096 $\rightarrow$ 1024 blocos 512 conjuntos $\rightarrow$ 9 bits para conjunto 2

Portanto são 9 bits para conjunto

Portanto são 2 bits para palavra

Portanto tag = 7 bits

Exercício 1 do Slide eliseu

a) $8G = 2^3 \times 2^{30} \rightarrow \frac{2^{33}}{2^1} \rightarrow 2^{32}$ ou seja 32 bits para endereçar.
 $2^1 \rightarrow$ Palavras endereçamento 16 bits.

b) São 64 linhas // 4 entradas por conjunto = 16 conjuntos.

Portanto 4 bits representam o conjunto.

2^7 de Tamanho de linha $\rightarrow 2^6$ palavras por bloco.

2^1 Tem de Palavra

6 bits para escrever então Palavra

$34 - 6 - 4 \rightarrow 14$ bits para tag

$$TAG = 14 \mid CONJUNTO = 4 \mid PALAVRA = 61 - 2$$

c) 2^{28} tamanho total $\rightarrow \frac{2^{28}}{2^4} \rightarrow 2^{24}$ blocos $\rightarrow \frac{2^{24}}{2^4} \rightarrow 2^{20}$ conjuntos $\rightarrow$ disputando um mesmo conjunto

tilibra

d) Seria maior pois agora seria 6 para linha // 6 palavra // 20 TAG.